Email Retrieval System

A hybrid RAG pipeline over email, with a SQL/semantic query router.

Scoped to fill the one gap in an existing portfolio · 5 weeks · $0 · 26 July 2026

1. Why this project, and why only this

Scoped against what the portfolio already contains. The point is to fill a gap, not to add a fifth broad project.

Skill	Status	Already covered by
Classical ML	Covered ×2	Security Intelligence (XGBoost, CUSUM) · Rainfall (Isolation Forest, DBSCAN, Z-score)
Deep learning / distillation	Covered	Knowledge-distillation framework — a stronger artifact than a one-off LoRA run
LLM applications	Covered	KuChiKu — Gemini, VAPI voice agents, orchestration
Full-stack / dashboards	Covered ×3	Next.js in three projects, Streamlit in one
Backend engineering	Covered	Spring Boot internship
RAG / information retrieval	Absent	**— nothing**

DELIBERATELY CUT FROM THE EARLIER PLAN

The noise classifier — a fourth trained classifier after XGBoost and an ensemble anomaly detector proves nothing new. Keep a 20-line rules filter for cost control; do not make it a showcase.

The LoRA distillation — redundant against an existing distillation *framework*, which is the better artifact.

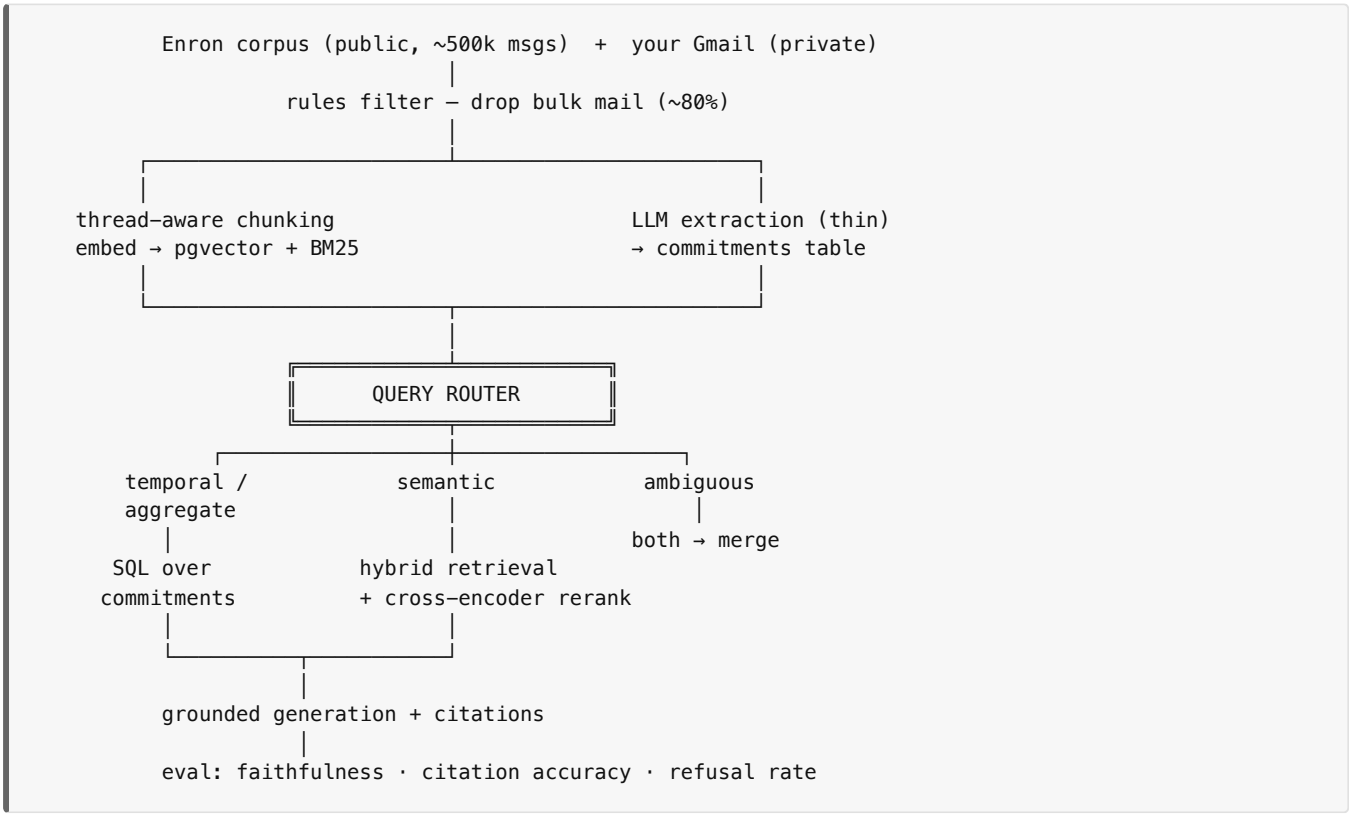
The portfolio is already broad. A fifth broad project dilutes it; one deep, narrow, heavily measured project is the counterweight. So **retrieval is ~60% of the effort here** and everything else exists to serve it.

2. What this project proves

Capability	Demonstrated by
Retrieval architecture	Sparse / dense / hybrid fusion, cross-encoder reranking
IR evaluation	Hand-verified eval set; recall@k, MRR, nDCG@10
Systematic ablation	Six independent dimensions, each measured in isolation
Engineering judgment	Published negative results; latency/quality tradeoffs made explicit
Hybrid symbolic + neural design	Query router sending temporal queries to SQL, semantic to vectors
Generation quality	Groundedness, citation accuracy, refusal rate on unanswerable queries
Meta-evaluation	LLM judge calibrated against human labels (Cohen's κ)

> **THE ONE THING THAT SEPARATES THIS FROM A TUTORIAL**
>
> Not the technology list — **every claim carries a number, and the eval set is published so anyone can reproduce it.** "Built RAG over my email" is indistinguishable from a weekend follow-along. An ablation table across six dimensions with a documented metric choice is not.

3. System design



Note the inversion from the earlier plan. Extraction is no longer the headline — it exists to populate the SQL side of the router, which is what makes the routing result provable. Two extraction models, not five.

4. Week 0 — Corpus and eval set

Nothing downstream is meaningful without this. If the project dies, it dies here.

Corpus

- **Enron** (CMU maildir) — public, so the demo and benchmark are shareable
- **Your Gmail** — via own OAuth client in testing mode; private, used for the qualitative half
- Rules filter: `List-Unsubscribe` header, `no-reply@` / `notifications@` patterns. ~20 lines, no ML

The retrieval eval set — 80 queries

Generate candidate queries with an LLM from real threads, then **hand-verify every single one**. For each query, record the set of message IDs that genuinely answer it.

Stratify deliberately, because the router result depends on this split:

Query class	Count	Example
Semantic / topical	35	"what did we decide about the pricing model"
Temporal / aggregate	25	"what's due this Thursday", "how many deadlines next week"
Entity-scoped	10	"everything from Priya about the audit"
Unanswerable (control)	10	Questions the corpus genuinely cannot answer

The unanswerable set is the one most people skip. It is how you measure whether the system correctly *refuses* instead of hallucinating — and refusal rate is a real, reportable metric.

Budget ~4 hours. Publish it as JSONL in the repo.

5. Weeks 1–2 — The retrieval ablations

Six dimensions, each varied independently against the same eval set. All of it runs free and locally.

Dimension 1 — Chunking

Strategy	Why include it
Fixed 512 tokens	The naive default everyone ships
Fixed 512 + 64 overlap	Isolates the effect of overlap alone
Thread-aware (never split mid-message)	Respects the natural document boundary
Whole message, no chunking	Most emails are short — test whether chunking helps at all

Almost nobody ablates chunking, and it frequently matters more than the retriever choice. Cheap to test, and a genuinely differentiating result.

Dimension 2 — Embedding model

`all-MiniLM-L6-v2` (22M) · `bge-small-en-v1.5` (33M) · `bge-base-en-v1.5` (109M) · Gemini embeddings (free tier). Report quality against index size and encode latency — a size/quality curve, not a single winner.

Dimension 3 — Retriever

Configuration	R@5	R@20	MRR	nDCG@10	p95 ms
BM25 only					
Dense only					
Hybrid — RRF fusion					
Hybrid — weighted score fusion					

Dimension 4 — Reranking

`bge-reranker-base` as a cross-encoder over the top-50, cut to top-5. Local, free. Report the nDCG gain *against* the added latency — this is the clearest tradeoff in the whole system.

Dimension 5 — Query transformation

Raw query · HyDE · multi-query expansion · decomposition for multi-hop questions.

> **PUBLISH THE ONES THAT LOSE**
> "HyDE cost 400 ms for +0.01 nDCG, so I cut it" is a *stronger* signal than shipping it because a blog post recommended it. Negative results with numbers demonstrate more judgment than a longer list of techniques.

Dimension 6 — Failure analysis

Take every query where recall@20 misses and categorize *why*:

- **Vocabulary mismatch** — query and document use different words for the same thing
- **Chunk boundary** — the answer straddles two chunks
- **Temporal** — no amount of semantic similarity answers "due Thursday" (this is what motivates the router)
- **Multi-hop** — the answer requires joining two messages
- **Genuine absence** — the eval label is wrong; fix it

A README section titled **"Where this fails"** is worth more than another 0.02 nDCG. It is also the section that generates the best interview conversation.

6. Week 3 — Extraction and the router

Extraction — deliberately thin

This is infrastructure for the router, not the centerpiece. Two models only: local Qwen 2.5 3B via Ollama, and Claude Haiku 4.5 batched as the quality ceiling.

```
commitments(
  id, source_msg_id, thread_id,
  type,          -- meeting | deadline | ask_of_me | promise_i_made
  counterparty,
  description,
  due_at,        -- absolute UTC, resolved from relative language
  confidence,
  status,        -- open | done | cancelled
  created_at
)
```

The one hard part worth measuring: **relative-date resolution**. "Next Thursday" in an email sent on a Tuesday. Pass the sent-timestamp and sender timezone into every prompt, require an absolute output, and report date-accuracy as its own metric. It is the top source of silent wrongness.

The query router — the differentiator

Classify each query into **temporal/aggregate → SQL**, **semantic → hybrid retrieval**, or **ambiguous → run both and merge**. Start with a small classifier or a cheap LLM call; measure either way.

Two things to measure:

1. **Router accuracy** against your labeled query classes
2. **End-to-end quality, router vs. vector-only**, broken down *by query class*

The expected headline — that dense retrieval fails almost completely on the 25 temporal queries while SQL answers them exactly — is the concrete, measured insight that makes this project memorable. It also justifies the whole two-store architecture with evidence rather than assertion.

Query class	Vector-only nDCG@10	With router	Δ
Semantic / topical			≈ 0 (same path)
Temporal / aggregate			large
Entity-scoped			

7. Week 4 — Generation evaluation

Most RAG projects stop at retrieval. Measuring the *answer* is where the remaining differentiation is.

- **Groundedness / faithfulness** — is every claim supported by a retrieved chunk?
- **Citation accuracy** — does each citation actually point at the source of that claim?
- **Refusal rate** — on the 10 unanswerable controls, does it correctly decline instead of inventing an answer?
- **Answer relevance** — does it address what was asked?

Calibrate the judge

Score with an LLM judge, then hand-label 50 of the same answers yourself and report **judge/human agreement (Cohen's κ)**.

Using an LLM judge shows competence. *Validating* it shows you understand that an unvalidated judge is just another unmeasured component. Very few candidates do this, and it is a strong signal.

8. Schedule

> **SHIP LINE**
> Five weeks, and **week 5 ends with a published repo**. Resist adding scope — this project's value is depth on one axis, not breadth.

Week	Deliverable
0	Repo, Enron loader, rules filter, **80-query eval set hand-verified and published**
1	Index built. Chunking ablation (4 configs) + embedding-model comparison (4 models)
2	Retriever ablation, reranking, query transformation. **Failure analysis**
3	Extraction (2 models) → commitments table. Query router + per-class results
4	Generation eval: groundedness, citations, refusal. Judge calibration (κ)
5	HF Spaces demo, README with all tables, `EVALUATION.md`, writeup on the router finding

9. Repo standards

- **README opens with the ablation tables.** Not an architecture diagram. Numbers first, prose second.
- `make bench` **reproduces every table** from the published eval set. Reproducibility is the entire claim.
- **Publish the eval set** — 80 queries with relevance labels, as JSONL. A public IR eval set is a real contribution; most people never build one.
- `EVALUATION.md` — why nDCG@10 and not accuracy, why these query classes, what the unanswerable controls measure. *This is the document that gets you hired.*
- **A "Where this fails" section** with the failure taxonomy and counts.
- **GitHub Actions CI** — free; runs tests and the local-model benchmark rows on push.
- **Demo on Hugging Face Spaces** (free) — query the Enron index live, see retrieved chunks with scores, the router's decision, and the cited answer. Public data, genuinely shareable.
- **One writeup** on the router finding or the chunking result. A single good post beats five thin ones.

10. Cost

Item	Cost
Embeddings + reranker — local sentence-transformers	$0
Qwen 2.5 3B extraction — Ollama, local	$0
Groq / Gemini AI Studio free tiers (router, judge)	$0
Postgres + pgvector — Neon free tier	$0
Hosting — Hugging Face Spaces free tier	$0
GitHub + Actions	$0
Claude Haiku ceiling rows — batched + cached	~$3, inside the signup credit
Total	**$0**

11. The resume bullets this produces

> ***Email Retrieval System*** *— github.com/you/project*
>
> *· Built a hybrid retrieval pipeline (BM25 + dense + cross-encoder rerank) over 500k emails; +0.21 nDCG@10 over a dense-only baseline at 180 ms p95*
>
> *· Ablated chunking, embedding model, fusion, and query transformation across an 80-query hand-labeled eval set — found thread-aware chunking worth +0.08 nDCG and HyDE not worth its 400 ms*
>
> *· Designed a SQL/semantic query router after measuring that dense retrieval answered only 12% of temporal queries correctly versus 96% via structured query*
>
> *· Evaluated generation for groundedness, citation accuracy, and refusal on unanswerable queries; validated the LLM judge against human labels at $\kappa = 0.81$*
>
> *· Published the eval set and a reproducible benchmark harness*

Five bullets, all retrieval, every one with a number. Paired with the existing ML, distillation, and LLM-application projects, the portfolio then covers classical ML, deep learning, LLM apps, IR/RAG, and backend — with no redundancy.

12. Appendix — higher-leverage than this project

Worth stating plainly: **retrofitting numbers onto the three existing projects will improve the resume more per hour than building anything new.** The models are already built. All that is missing is running evaluation and writing down the results.

Current bullet	What it should say
~~Developed backend machine learning models in Python using XGBoost and CUSUM-based anomaly detection to identify real-time and sophisticated network threats.~~	XGBoost threat classifier at **0.94 AUC / 2.8% FPR** over **1.2M network flows**; CUSUM drift detector caught **89%** of injected attacks within 4 windows at **12 ms p95** inference.
~~Implemented ensemble anomaly detection using Isolation Forest, DBSCAN, and Rolling Z-Score Analysis for regional risk assessment.~~	Ensemble of Isolation Forest, DBSCAN and rolling Z-score raised anomaly **F1 from 0.61 to 0.78** over the best single detector across **N years / M stations** of weather data.
~~Implemented AI-driven interview evaluation with personalized feedback, performance analytics, and progress tracking.~~	Interview scoring rubric validated against **N human-rated sessions** (κ = X); **p50 voice round-trip latency Y ms** across **Z sessions**.

Every one of those numbers is recoverable from work already done. Budget one weekend.

One structural note: **Security Intelligence and Rainfall are both "anomaly detection plus a dashboard."** A reader sees the same shape twice. Lead with the stronger one and compress the other, or reframe one around what makes it distinct — real-time streaming versus geospatial forecasting.